ZIM School - Use Cases

04. Learners: enrollment, learner records, transfers, SF1 and learner IDs

Modules: Learners
11 use cases (changes) · 8 views and reports
Written from the use case commands, handlers, policies and rules in `src/` on 2026-11-10.

1. What this area does

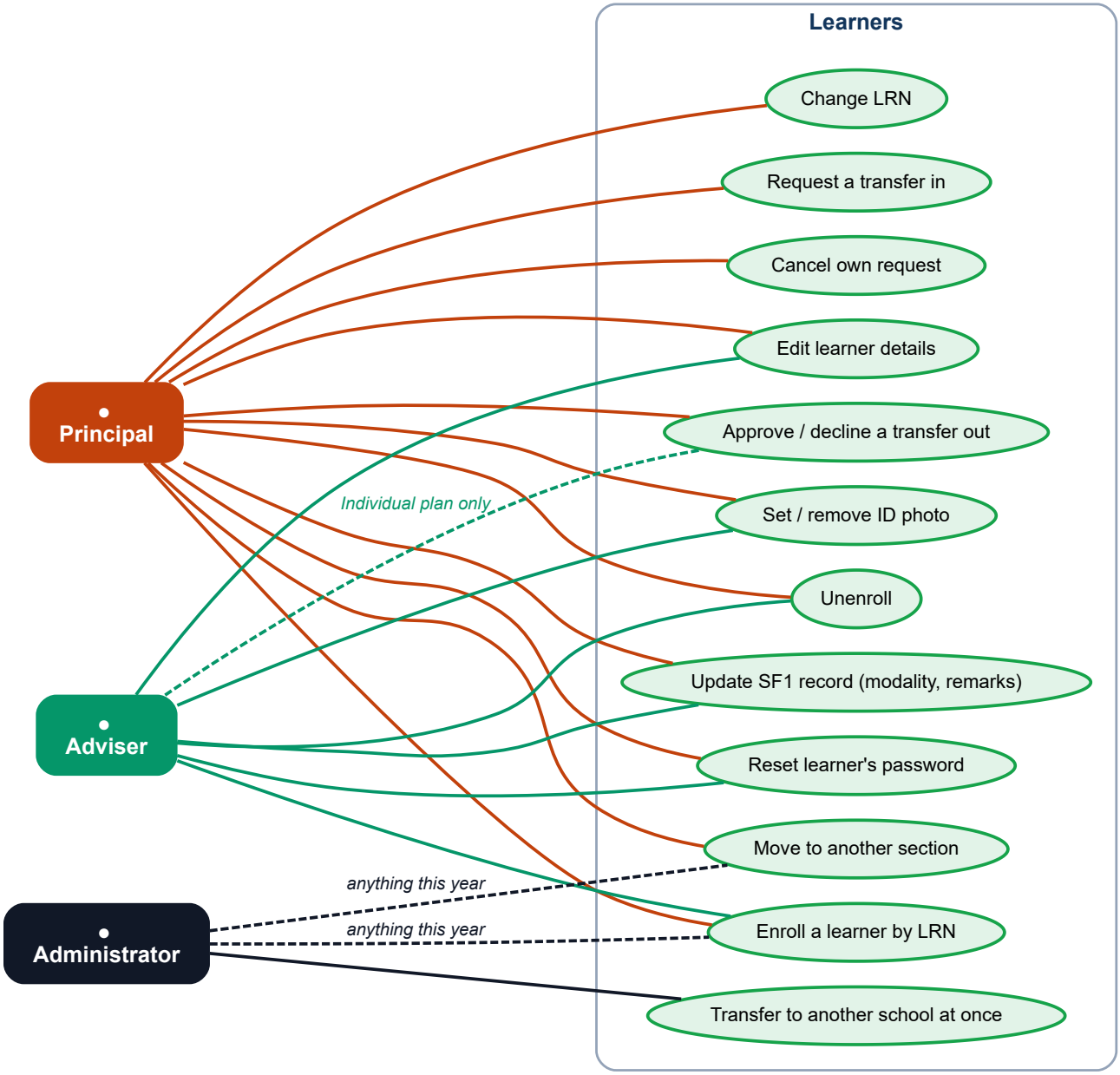
A **learner** is kept for life and identified by the 12-digit **LRN**; each school year the learner gets one **enrollment** in a section. The school's pages (`/schools/{id}/learners`, used by the principal and administrators) and the adviser's class pages (`/teacher/class`) share the same use cases; on the adviser's pages the section is always their own.

Who may do what (`LearnerPolicy`): the school's principal may do everything for the school this year; the section's adviser may enroll into, edit, reset passwords and unenroll in their own section, but may not change an LRN or move learners; administrators may do anything this school year and transfer learners directly. Nobody may change anything without a current school year, and past years are read-only.

Every change takes the `learner-<id>` lock (and `learners-school-<id>` where seats are counted) before reading anything. Student counts are capped by the plan; a learner with a pending transfer can't be changed.

2. Use case map

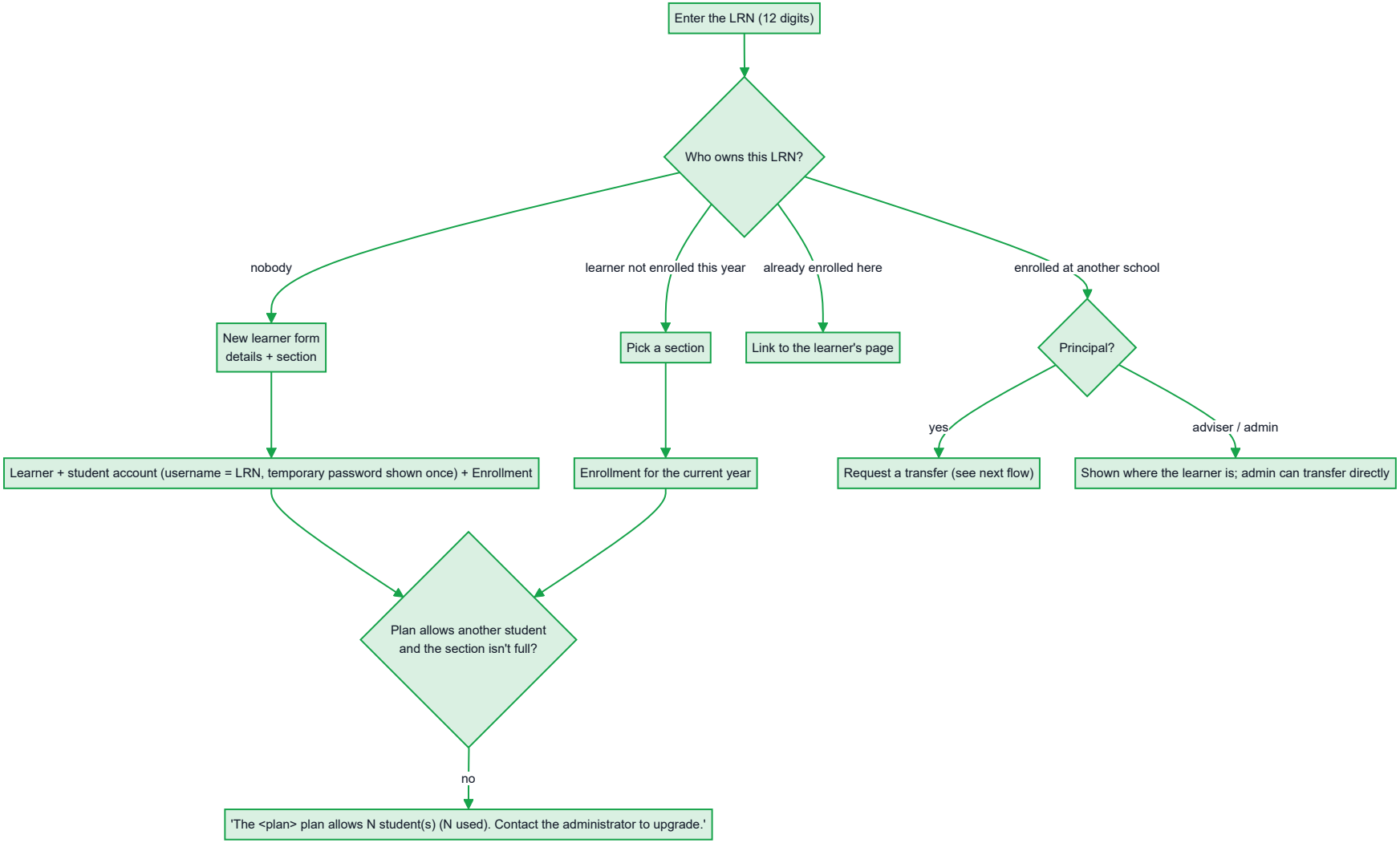
A standard use case diagram: the actors on the left, each use case as an ellipse inside the system frame, and a line for each use case an actor starts. A dashed line has a note on when it applies.



Principal · Adviser · Administrator — Line colour = the actor who starts it.

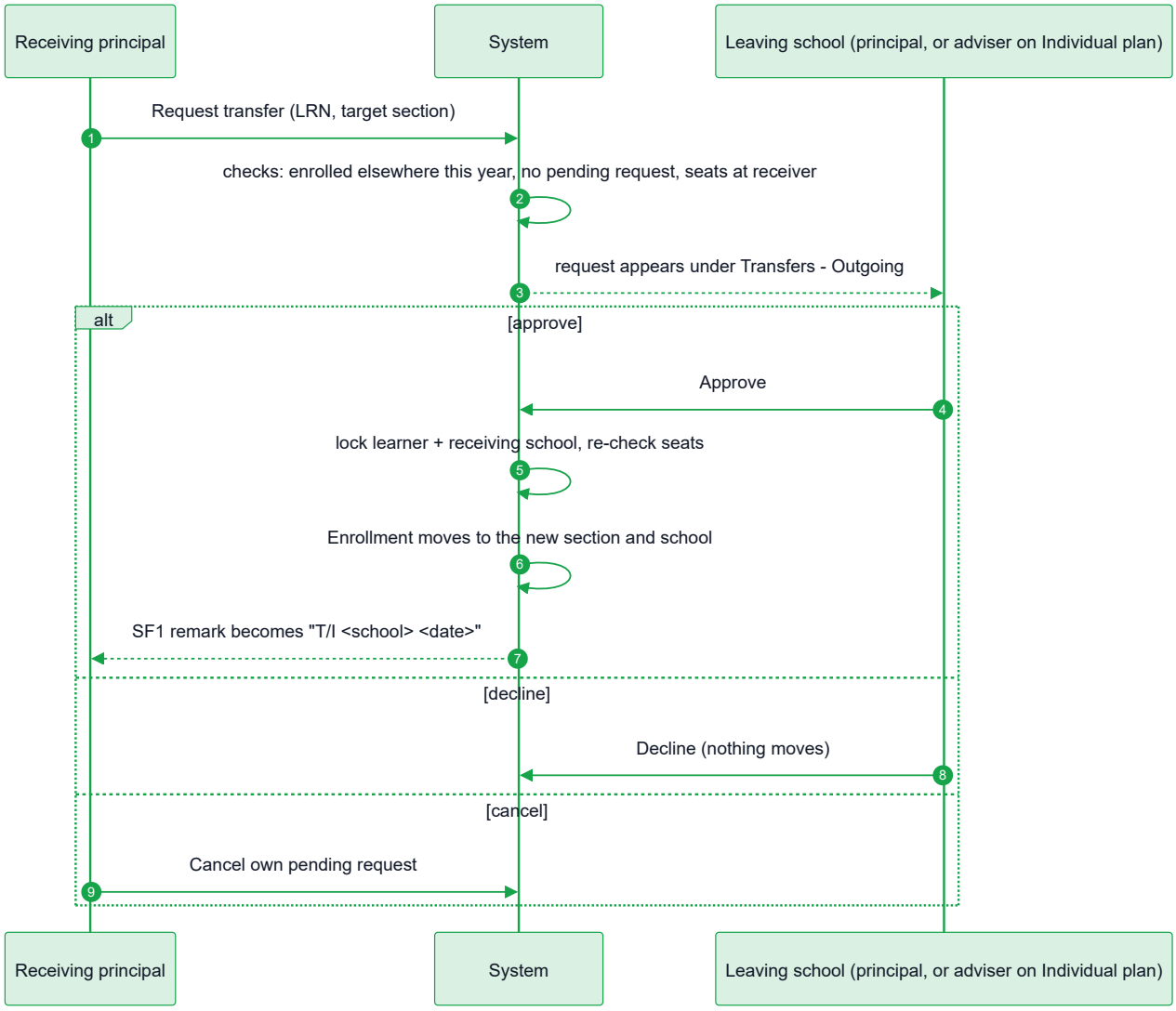
Learners — Ellipse colour = the module the use case belongs to.

3.1 Flow: Enrolling by LRN



● **Learners** Drawn in the colour of the module this flow belongs to.

3.2 Flow: Transfer between schools



Learners Drawn in the colour of the module this flow belongs to.

4. Use cases (changes)

Each card is one use case command sent through the use case bus. It runs in one database transaction: if any rule refuses, nothing is saved and the message in *Alternatives and refusals* is shown to the user.

LRN-01 Enroll a learner

Primary actor	Principal, Adviser, Administrator
Goal	Put a new or returning learner into a section for the current school year.
Where (route)	`school_learners_enroll /schools/{id}/learners/enroll; teacher_class_enroll /teacher/class/enroll`
Preconditions	• May enroll into the section (principal of the school, adviser of the section, or admin); current school year.
Main flow	1. The user types the LRN; the page looks it up (new / not enrolled / enrolled here / enrolled elsewhere). 2. New learner: fill in the learner details (names, sex, birth date, address, parents, guardian, contact) and choose the section. 3. Returning learner: choose the section only. 4. The system takes the learner and school locks, checks the plan's student limit and section capacity, and saves. 5. A new learner gets a student account (username = LRN) with a temporary password that is shown once.
Alternatives and refusals	• "The LRN must be exactly 12 digits." / "LRN <lrn> belongs to <name>." • "Birthdate must be before today." • "The username <lrn> is already taken by another account. Ask the administrator." • "Choose a section of the current school year." • Plan limit: "The <plan> plan allows N student(s) (N used). Contact the administrator to upgrade." • Busy: "Another change is being saved. Please try again."
Result (data changed)	Learner + User (student) for a new learner; Enrollment; activity log "Enrolled <LRN> in <section>".
In the code	`EnrollLearnerCommand (LearnerDetails) → LearnerEnrollment::enrollNew / enrollExisting`

LRN-02 Edit learner details (and the LRN)

Primary actor	Principal, Adviser, Administrator
Goal	Correct the learner's personal data.
Where (route)	`school_learners_edit; teacher_class_edit`
Preconditions	• The learner is enrolled in a section the user may edit; no pending transfer.
Main flow	1. The user edits the details and saves. 2. Only principals and admins see the LRN field; the student's username follows a changed LRN.
Alternatives and refusals	• LRN format / LRN taken. • Username taken by another account. • Pending transfer or plan blocked.
Result (data changed)	Learner (and User.username when the LRN changes).
In the code	`UpdateLearnerCommand`

LRN-03 Update this year's SF1 record

Primary actor	Principal, Adviser, Administrator
Goal	Record the learning modality and the SF1 remark (e.g. dropped, late enrollment, CCT, Balik-Aral, SNED, accelerated).
Where (route)	`school_learners_enrollment; teacher_class_enrollment`
Preconditions	• The learner is enrolled this year and editable.
Main flow	1. The user picks the modality and, if needed, a remark code with details and date. 2. Saved on the enrollment.
Alternatives and refusals	• Details without a code: "Choose a remark for these details."
Result (data changed)	Enrollment.learning_modality, remark_code, remark_details, remark_date.
In the code	`UpdateEnrollmentRecordCommand`

LRN-04 Set or remove the learner's ID photo

Primary actor	Principal, Adviser, Administrator
Goal	Add the photo that prints on the learner ID card.
Where (route)	`school_learners_photo / _photo_remove; teacher_class_photo / _photo_remove`
Preconditions	• The learner is editable.
Main flow	1. The user uploads an image; it is stored as WebP (or removed).
Alternatives and refusals	• Image not readable or too large.
Result (data changed)	Learner's account photo.
In the code	`UpdateLearnerPhotoCommand`

LRN-05 Reset a learner's password

Primary actor	Principal, Adviser, Administrator
Goal	Help a student who forgot their password.
Where (route)	`school_learners_reset_password; teacher_class_reset_password`
Preconditions	• The learner is editable.
Main flow	1. The user presses Reset password; a temporary password is shown once.
Alternatives and refusals	• -
Result (data changed)	User.password, must_change_password.
In the code	`ResetLearnerPasswordCommand`

LRN-06 Unenroll a learner

Primary actor	Principal, Adviser, Administrator
Goal	Take a learner off the class list (e.g. enrolled by mistake).
Where (route)	`school_learners_unenroll; teacher_class_unenroll`
Preconditions	• The learner is enrolled this year; no pending transfer.
Main flow	1. The user confirms Unenroll. 2. The enrollment of the current year is removed; the learner, the account and past years stay.
Alternatives and refusals	• Pending transfer / plan blocked.
Result (data changed)	Enrollment removed.
In the code	`UnenrollLearnerCommand`

LRN-07 Move a learner to another section of the same school

Primary actor	Principal, Administrator
Goal	Change a learner's section within the school.
Where (route)	`school_learners_move /schools/{id}/learners/{learner}/move`
Preconditions	• Principal of the school or admin; target section in the same school and year.
Main flow	1. The principal picks the new section. 2. The system checks capacity and moves the enrollment.
Alternatives and refusals	• "Choose a section of the same school." • Section full / plan limit.
Result (data changed)	Enrollment.section_id.
In the code	`MoveLearnerCommand`

LRN-08 Transfer a learner to another school directly

Primary actor	Administrator
Goal	Move a learner to a section of another school at once (no request needed).
Where (route)	`school_learners_transfer /schools/{id}/learners/{learner}/transfer`
Preconditions	• Administrator; the target section is at another school this year.
Main flow	1. The admin picks the school and the section on the learner's page. 2. The system takes the learner and receiving school locks, checks the receiving plan and moves the enrollment.
Alternatives and refusals	• "This section is not at another school this school year." • "Choose a section of another school." • Receiving school's plan limit.
Result (data changed)	Enrollment.section_id and school_id.
In the code	`TransferLearnerCommand`

LRN-09 Request a transfer in

Primary actor	Principal (receiving school)
Goal	Ask the learner's current school to let them move into one of your sections.
Where (route)	`school_transfers_request /schools/{id}/transfers/request`
Preconditions	• Principal (not admin) of the receiving school; the LRN belongs to a learner enrolled at another school this year; no pending request.
Main flow	1. On the enroll page the LRN shows 'enrolled at <school>'; the principal chooses a section and sends the request. 2. The system re-reads the enrollment under the learner lock and checks the receiving school's seats.
Alternatives and refusals	• "This learner is no longer enrolled at another school." (changed meanwhile) • "Choose a section of another school." • Plan limit at the receiving school. • Already a pending transfer.
Result (data changed)	TransferRequest (status pending).
In the code	`RequestTransferCommand → TransferRequests::request`

LRN-10 Approve or decline a transfer out

Primary actor	Principal (school being left); adviser on the Individual plan; Administrator
Goal	Decide whether a learner may leave for the requesting school.
Where (route)	`school_transfers_approve / _decline (/schools/{id}/transfers/{transfer}/...)`
Preconditions	• The request is pending and from this school's section this year.
Main flow	1. The user opens Transfers - Outgoing and presses Approve or Decline. 2. Approve: the system locks the learner and the receiving school, re-checks the receiver's seats, moves the enrollment to the requested section and marks the request approved. 3. Decline: the request is marked declined; nothing moves.
Alternatives and refusals	• "This request was already decided." • "This learner is no longer enrolled." • Receiving plan limit at approval time.
Result (data changed)	TransferRequest.status, decided_by, decided_at; on approval Enrollment moves (SF1 remark T/I at the new school).
In the code	`ApproveTransferCommand, DeclineTransferCommand`

LRN-11 Cancel a transfer request

Primary actor	Principal (receiving school)
Goal	Withdraw a request the school sent.
Where (route)	`school_transfers_cancel /schools/{id}/transfers/{transfer}/cancel`
Preconditions	• The request is pending and was sent to this school.
Main flow	1. The principal presses Cancel under Transfers - Incoming.
Alternatives and refusals	• "This request was already decided."
Result (data changed)	TransferRequest.status = cancelled.
In the code	`CancelTransferCommand`

5. Views, downloads and reports (read only)

Each row is one query sent through the query bus. Queries change nothing.

Page / file	Who	Route	What it shows	Query
School's learners	Principal, Administrator	`school_learners` `/schools/{id}/learners` `(principal_learners shortcut)`	Paged learners this year, search by LRN or name, filter by section.	`ListLearnersCommand`
Class list	Principal, Adviser	`school_learners_class;` `teacher_class` `/teacher/class`	A section's learners, boys first.	`GetClassListCommand,` `GetAdvisoryCommand`
Enroll page	Principal, Adviser	`school_learners_enroll;` `teacher_class_enroll`	Lookup result for an LRN and the sections to choose.	`GetEnrollPageCommand`
Learner page	Principal, Adviser, Administrator	`school_learners_show;` `teacher_class_show`	Details, this year, history, transfers, and what the user may do.	`GetLearnerCommand,` `GetLearnerFormCommand,` `GetEnrollmentRecordCommand`
Transfers (incoming / outgoing)	Principal, Adviser	`school_transfers` `/schools/{id}/transfers`	Requests this year; an adviser sees only their section's outgoing ones.	`ListTransfersCommand,` `GetTransferCommand,` `GetTransferRequestFormCommand`
SF1 School Register (.xls)	Principal, Adviser	`school_learners_sf1;` `teacher_class_sf1`	The section's SF1 filled from the enrollments (PhpSpreadsheet).	`GetSf1FileCommand`
Learner ID cards (PDF)	Principal, Adviser	`school_learners_ids /` `_id; teacher_class_ids` `/ _id`	One card or a section's cards with photo and QR code.	`GetSectionIdCardsCommand,` `GetLearnerIdCardCommand`
Admin learner search	Administrator	`admin_learners` `/admin/learners`	Learners across schools by LRN or name, with their current or last enrollment.	`ListAdminLearnersCommand`